j4_stepper_neck

Neck stepper driver board for the Johnny 4 robot, and the middle node of the stepper daisy chain. Runs on an ESP32-DEV (WROOM) module. It drives the three neck steppers via STEPPERONLINE DM556TE digital step/dir drivers, relays the eye-pop value down to j4_stepper_eyes, and reports neck/eye health back up to the receiver. Step pulses run on the ESP32 RMT peripheral via FastAccelStepper, so the CPU never blocks.

The two stepper boards were briefly consolidated onto one ESP32 (the repo was named j4_stepper for that stretch). They are split again as of v0_12: with the eyes on their own board there are enough pins to give **every limit switch its own input**, so the firmware knows exactly which endpoint was hit.

Daisy chain

```
j4_controller --ESP-NOW--> j4_receiver (TTGO)
                                |  UART1 (2-way, 250000)
                                v
                          j4_stepper_neck   (3x DM556TE)   <-- this board
                                |  UART2 (2-way, 250000)
                                v
                          j4_stepper_eyes   (2x TMC2209)
```

The TTGO only has one free hardware UART, so the boards are chained rather than each wired to the TTGO. This board is the relay in the middle: it takes the full packet from the receiver, keeps the neck fields, and passes the eye field downstream; status flows back the other way.

What this board does

- Receives `nL,nR,nP,eye` from j4_receiver over UART1 (2-way)
- Drives 3 neck steppers via STEPPERONLINE DM556TE drivers (neck left, neck right, neck pivot)
- Forwards the `eye` value to j4_stepper_eyes over UART2
- Monitors six limit switches -- one per axis endpoint -- so faults name the exact switch (`NL MIN` , `NP MAX` , ...)
- Merges its own limit status with the eyes board's status and a link-alive check, and reports a single status string back upstream
- Reports the eyes-board link state as a separate `EY:<0/1>` line so the connection screens can show j4_stepper_eyes
- Filters out tiny commanded changes (per-axis threshold) so the motors do not jitter at rest

The DM556TE is a standalone driver: motor current and microstep resolution are set by DIP switches on the driver itself, and there is no telemetry bus back to the ESP32. Neck faults therefore come from the limit switches.

Hardware

Component	Details
Microcontroller	ESP32-DEV (WROOM-32)
Neck drivers	3x STEPPERONLINE DM556TE (digital step/dir, opto-isolated inputs)
Level shifters	2x SN74AHCT125N quad buffers (3.3V STEP/DIR up to 5V)
Neck motors	STEPPERONLINE 17HE24-2104S (2.1A)
Motor supply	24-50VDC to each DM556TE +Vdc

Limit switches	6x normally-closed, one per axis endpoint

Link protocol

Upstream (UART1, to/from j4_receiver):

Direction	Message	Meaning
in	`nL,nR,nP,eye\n`	neck L/R/pivot + eye-pop, all 0-3200
in	`CAL\n`	run a homing/calibration pass (also forwarded to the eyes board)
out	`ST:<text>\n`	merged status, e.g. `ST:OK`, `ST:NL MIN`, `ST:NP MAX`, `ST:EYES OFFLINE`
out	`EY:<0/1>\n`	eyes-board link state (1 = alive)

Downstream (UART2, to/from j4_stepper_eyes):

Direction	Message	Meaning
out	`<eye>\n`	the forwarded eye-pop value (0-3200)
out	`CAL\n`	tell the eyes board to calibrate too
in	`S:<text>\n`	the eyes board's status (`S:OK`, `S:EL MIN`, ...)

The parsers use fixed char buffers (no Arduino `String`, so no heap fragmentation). A line that does not parse is ignored.

Status reporting

Every 200 ms the board sends one status string upstream, chosen by priority:

1. `EYES OFFLINE` -- no status from the eyes board within 800 ms
2. a neck limit fault naming the exact switch -- `NL` / `NR` / `NP` + `MIN` (home end) or `MAX` (far end); a switch is engaged or its wire is broken
3. an eye fault relayed up from the eyes board (`EL` / `ER` + token)
4. `OK`

Limit switches and calibration

Each axis has **one normally-closed (NC) switch per endpoint** -- six inputs total. Each switch shorts its input to GND while the axis is in range: LOW = in range, HIGH = that endpoint was hit **or** its wire broke (fail-safe). `MIN` is the home end that `CAL` drives toward; `MAX` is the far end.

- The three MIN inputs sit on the input-only pins 34/36/39, which have no internal pull-up: each needs an **external 10k to 3.3V**.
- The three MAX inputs sit on ordinary GPIOs 18/19/23 and use the **internal pull-up** -- no external resistor needed.

In normal operation the safety check is direction-aware: an axis is stopped only when it is moving *toward* a tripped endpoint, so a stopped axis can always drive itself back off a limit.

On a `CAL` command each axis homes toward its MIN switch. An axis already sitting on the switch is detected immediately (the switch reads pressed) and just backs off and zeros -- the per-endpoint inputs make stall detection

unnecessary on this board. Hitting the MAX switch during homing stops the axis (wrong direction or a wiring fault). The homing speed, back-off, and timeout are `#define`s at the top of `main.cpp` and **need tuning on the real mechanism**.

DM556TE driver wiring (three identical)

Signal inputs (opto-isolated, via the 74AHCT125 buffers)

The DM556TE inputs are optically isolated and rated for 5-24V logic at 7-20mA. The ESP32's 3.3V is at the low edge of that spec, so the six STEP/DIR signals pass through two **SN74AHCT125N** quad buffers (next section), which shift them to a full 5V. The drivers stay wired **common-cathode / active-high**, so the pulse/direction polarity stays normal and the firmware is unchanged:

DM556TE terminal	Connect to
PUL+	74AHCT125 Y output carrying that axis's STEP
PUL-	ESP32 GND
DIR+	74AHCT125 Y output carrying that axis's DIR
DIR-	ESP32 GND
ENA+ / ENA-	leave unconnected (driver is enabled by default)

SN74AHCT125N level shifters (U1 + U2)

Quad bus buffer, non-inverting, DIP-14. The AHCT family has TTL input thresholds (a high is anything above 2.0V), so the ESP32's 3.3V logic drives the inputs directly -- while the outputs swing to VCC = 5V. Two chips cover the six STEP/DIR signals with two buffers to spare.

- **VCC (pin 14)** -> the ESP32 board's 5V pin; **GND (pin 7)** -> common ground
- **Every `/OE` pin (1, 4, 10, 13)** -> GND, so all outputs are permanently enabled
- **100nF decoupling cap** across VCC/GND of each chip, close to the pins
- **Unused inputs must not float** -- tie U2's 3A/4A to GND, leave 3Y/4Y unconnected

Chip pin diagram

```
       SN74AHCT125N (DIP-14, top view, notch at the top)

                        +---U---+
     1/OE (to GND) -- 1|       |14 -- VCC (5V)
     1A  (input)   -- 2|       |13 -- 4/OE (to GND)
     1Y  (output)  -- 3|       |12 -- 4A  (input)
     2/OE (to GND) -- 4|       |11 -- 4Y  (output)
     2A  (input)   -- 5|       |10 -- 3/OE (to GND)
     2Y  (output)  -- 6|       | 9 -- 3A  (input)
     GND           -- 7|       | 8 -- 3Y  (output)
                        +-------+
```

Buffer assignments

U1 (neck-left + neck-right):

Buffer	Input (A) from	Output (Y) to
1 (pins 2 -> 3)	ESP32 26 (STEP NL)	neck-left DM556TE PUL+
2 (pins 5 -> 6)	ESP32 25 (DIR NL)	neck-left DM556TE DIR+
3 (pins 9 -> 8)	ESP32 33 (STEP NR)	neck-right DM556TE PUL+
4 (pins 12 -> 11)	ESP32 32 (DIR NR)	neck-right DM556TE DIR+

U2 (neck-pivot, two buffers spare):

Buffer	Input (A) from	Output (Y) to
1 (pins 2 -> 3)	ESP32 27 (STEP NP)	neck-pivot DM556TE PUL+
2 (pins 5 -> 6)	ESP32 14 (DIR NP)	neck-pivot DM556TE DIR+
3, 4	tie 3A/4A to GND (spare)	leave 3Y/4Y unconnected

Power and motor

DM556TE terminal	Connect to
+Vdc / GND	24-50VDC motor supply (its GND common with the ESP32 GND)
A+ / A-	stepper phase A
B+ / B-	stepper phase B

If an axis runs the wrong direction, swap that motor's A+/A- pair (or invert its DIR mapping in firmware). All grounds -- ESP32, driver signal, and motor supply -- must be common.

DIP switch settings

Current and microstepping are set on the driver, not in firmware.

- **Current (SW1-SW3) + SW4:** for the 2.1A neck motors use the **2.1A-peak / 1.5A-RMS** row: **SW1=on, SW2=off, SW3=off**, with **SW4=on** (full current). Move up one row if you need more holding torque and the motor stays cool.
- **Microstepping (SW5-SW8):** set **3200 pulse/rev** -> **SW5=on, SW6=on, SW7=off, SW8=on**. This makes one commanded unit in firmware equal one step pulse, matching the old 16-microstep scale so all the motion numbers carry over unchanged.

Pin assignments

GPIO	Function
26 / 25	neck-left STEP / DIR -> 74AHCT125 U1 -> DM556TE PUL+ / DIR+
33 / 32	neck-right STEP / DIR -> 74AHCT125 U1 -> DM556TE PUL+ / DIR+
27 / 14	neck-pivot STEP / DIR -> 74AHCT125 U2 -> DM556TE PUL+ / DIR+
4 / 13	Upstream UART1 RX / TX -- to j4_receiver: RX from TTGO GPIO 21, TX to TTGO GPIO 22

22 / 35	Downstream UART2 TX / RX -- to j4_stepper_eyes: TX to eyes GPIO 4, RX from eyes GPIO 13
34	neck-left MIN limit (input-only, external 10k pull-up)
36	neck-right MIN limit (input-only, external 10k pull-up)
39	neck-pivot MIN limit (input-only, external 10k pull-up)
18	neck-left MAX limit (internal pull-up)
19	neck-right MAX limit (internal pull-up)
23	neck-pivot MAX limit (internal pull-up)

The board is a bare ESP32-D0WD (LOLIN32-Lite style). It does **not break out GPIO 21**, so the downstream UART TX is on **GPIO 22** (which also drives the onboard LED, giving a handy downstream-activity blink). The downstream RX is on the input-only **GPIO 35**; the board has **no battery-sense pin**, so 35 is safe to use. The step/dir lines use plain bidirectional GPIOs (strapping pins 0/2/12/15 and flash pins 6-11 avoided). Pins 5/16/17 are free.

Limit switches are not required for normal operation, so you can wire them whenever you add them.

Pin diagram

This matches the real pin order on the board: hold it component-side up with the USB-C connector at the bottom. The two header rails read top-to-bottom as shown.

```
         LEFT rail                              RIGHT rail
                          +--------------+
  limit NR MIN -- 36 |    |              |    | 3V3   3.3V (limit pull-ups)
  limit NP MIN -- 39 |    |              |    | 22    DN UART2 TX -> eyes 4   (LED)
         reset -- EN |    |  ESP32-D0WD  |    | 19    limit NR MAX
  limit NL MIN -- 34 |    |  (LOLIN32-   |    | 23    limit NP MAX
DN RX <-eyes13 -- 35 |    |   Lite style)|    | 18    limit NL MAX
    DIR neck-R -- 32 |    |              |    | 5     (free)
   STEP neck-R -- 33 |    |              |    | 17    (free)
    DIR neck-L -- 25 |    |              |    | 16    (free)
   STEP neck-L -- 26 |    |              |    | 4     UART1 RX <- TTGO 21
   STEP neck-P -- 27 |    |              |    | 0     (free / BOOT)
    DIR neck-P -- 14 |    |              |    | 2     (free)
        (free) -- 12 |    |              |    | 15    (free)
        ground -- GND|    |              |    | 13    UART1 TX -> TTGO 22
                          +---[ USB-C ]--+

       SN74AHCT125N x2 (DIP-14, top view, notch at the top)

                         +---U---+
     1/OE (to GND) -- 1|       |14 -- VCC (5V)
     1A  (input)   -- 2|       |13 -- 4/OE (to GND)
     1Y  (output)  -- 3|       |12 -- 4A  (input)
     2/OE (to GND) -- 4|       |11 -- 4Y  (output)
     2A  (input)   -- 5|       |10 -- 3/OE (to GND)
     2Y  (output)  -- 6|       | 9 -- 3A  (input)
     GND           -- 7|       | 8 -- 3Y  (output)
                         +-------+
```

```
U1: 1A<-26 1Y->NL PUL+ ; 2A<-25 2Y->NL DIR+ ; 3A<-33 3Y->NR PUL+ ; 4A<-32 4Y->NR DIR+
U2: 1A<-27 1Y->NP PUL+ ; 2A<-14 2Y->NP DIR+ ; 3A,4A->GND (spare) ; 3Y,4Y n/c
GPIO 21 is NOT present on this board, which is why the downstream TX is on 22.
Neck STEP/DIR pins pass through the 74AHCT125 buffers (3.3V -> 5V) into the
DM556TE PUL+/DIR+ inputs (PUL-/DIR- to GND). 100nF across each chip's VCC/GND;
all /OE pins to GND.
Limits: one NC switch per endpoint. MIN ends (34/36/39) need external 10k
pull-ups to 3.3V; MAX ends (18/19/23) use the internal pull-ups.
```

Motion settings

Neck values match the originals: `setSpeedInHz(3200)`, `setAcceleration(16000)`, and a per-axis change threshold of 20. With each DM556TE set to 3200 pulse/rev, one commanded unit equals one pulse, so these are the same numbers as the old 16-microstep setup. The neck-left axis is mapped inverted (`(3200 - nL) / 2`).

Building and uploading

```
# Build
pio run

# Upload
pio run --target upload

# Monitor serial output
pio device monitor
```

Related projects

- **j4_receiver** -- the TTGO that streams `nL,nR,nP,eye` to this board and relays its status to the controller
- **j4_stepper_eyes** -- the eye-pop stepper board downstream of this one